Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

NodeSync: Fault-Tolerant Distributed Consensus Simulator for E-Commerce

Team Name: *Team # DSCPP-III-2026-T295*	*Beyond Binary*
Team member 1 (Team Lead)	*Aakshi – 2510380120* *2510380120@geu.ac.in* 
Team member 2	*Tripathi, Astik – 2510370444* *2510370444@geu.ac.in* 
Team member 3	*Sidharth – 2512510032* *2512510032@geu.ac.in* 
Team member 4	*Mittal, Sanjam – 2510370433* *2510370433@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

NodeSync FaultTolerant Distributed Consensus Simulator for E-Commerce is a simulation-based project designed to demonstrate how distributed systems maintain consistency and handle node or network failures.The project uses the Raft consensus algorithm to model multiple interconnected nodes that coordinate operations such as placing orders managing inventory and replicating logs.

Data Structures and Algorithms in C are used to organize data and messages while Object-Oriented Programming concepts in C++17 are used to model nodes, network communication storage, messages, and Raft states.load balancer is included to manage request routing,with leader-aware routing for write operations.

NodeSync shows how a cluster of nodes elects a leader, copies commands to every node and commits them once majority has them. Leader election, replication and majority commit already run in the simulator.Failure handling is the part we arre still building. The simulator can record message count, election time, consensus time and recovery time so we can see how the cluster behaves under load and after a crash. It is a learning tool for fault tolerant consensus, data structures and OOP in C++, not a production e commerce system.

Updated Project Approach and Architecture

The current project is based on developing Nodesync simulation as a distributed system that demonstrates fault-tolerant consensus in an ecommerce environment.

NodeSync is split into two layers. The lower layer is written in C and holds a linked list a queue the log structure which store the entries and messages the nodes work with The upper layer is C++ and holds the Node, RaftNode, Storage, Network Message classes. We used classes, encapsulation, constructors, composition enums to build it.

There are no real sockets here. The Network class hands Message objects from one node to another inside the same process. A message is a vote request a vote reply a heartbeat or a log entry. Raft picks the leader and keeps the nodes' logs consistent. Commands like adding a product or changing stock are the entries we replicate to show how the log works.

The architecture, the C data structures, the node and storage modules, the simulated network, leader election and basic log replication are all in place. The project builds with CMake and uses only the standard C and C++ libraries with no external dependencies and no machine learning.

Phase 3 is about failures - a crashed node, a crashed leader, a network partition and bringing a node back by syncing its log. We will also do more testing, measure performance and tune what we find.

Tasks Completed

Task Completed	Team Member
Implemented the C queue - developed the Message class and election timeout handling, contributed to AppendEntries message construction and stock-update commands, implemented network failure simulation and message-count metrics and developed the leader failure scenario.	**Aakshi**
Implemented the C log structure, developed the Network module, contributed to RequestVote and voting logic follower log consistency and e-commerce command validation, worked on leader-failure handling, consensus-time measurement and the node-failure scenario.	**Astik Tripathi**
Implemented linked-list operations in C; contributed to the Node class and Raft state definitions, implemented leader-side log appending and product -addition commands, worked on node failure handling, election-time measurement, and the normal-operation simulation scenario.	**Sidharth**
Integrated the C log/DSA layer with C++ Storage; developed communication integration test,; implemented majority-based leader election, heartbeat handling, log commitment, and application of committed commands, contributed to follower recovery, log catch-up, recovery-time measurement and the network partition scenario.	**Sanjam Mittal**

Challenges/Roadblocks

The first problem was joining the C data structures to the C++ classes. list, queue and log had to be easy to call from C++ without making the build complicated.

Raft itself was the second problem. Follower, candidate and leader states, majority votes and log replication all depends on several nodes interacting which is hard to picture at first. We broke it into small steps building election first and replication after and put them together once each part worked.

Third we had to fake a network, With no sockets allowed the Network class delivers messages in memory and acts as the wire between nodes.

Log synchronization after a failure is still open. If a node or the leader goes down the others must catch it up when it returns and this is not finished.

Finally each of us built our own module separately. so putting them together brought up mismatches.We compile and run integration tests regularly to catch these early The main architecture and implementation hurdles are behind us.
Fault tolerance and performance measurement with advanced testing are left for the next phase.

Tasks Pending

Task Pending	Team Member (to complete the task)
Enhance Raft leader election, heartbeat handling term management and majority based consensus logic and integrate the Raft workflow	Aakshi
Test vote handling, Log consistency, Network partitions and synchronization of logs after failures	Astik Tripathi
Validate node crash/restart behavior, Raft state transitions and leader re-election through normal and failure scenarios	Sidharth
Complete storage and Network integration and validate order and inventory updates through committed log entries replicated across nodes	Sanjam Mittal
Complete end-to-end integration of Storage, log replication, majority commits and recovery, verify performance metrics including election time, consensus time, recovery time and message count	All team members

Project Outcome/Deliverables

With the Raft consensus algorithm at its core, NodeSync is a simulator for an e-commerce system built to be fault tolerant. To date, the project has put forward the following deliverables:

- A modular architecture with DSA in C and OOP in C++.
- The underlying DSA work, covering the likes of linked list, queue and log implementations.
- C++ modules to handle the simulation of communication, storage, messages as well as the nodes and their Raft states.
- Simple majority-based leader election feature.
- Implementation of e-commerce commands such as product addition and stock updates.
- Basic log replication and majority-based commits.
- Project CMake configuration and well-organized source code.
- Initial simulation of communication and consensus among several nodes.

The end deliverable will be a console-based distributed consensus simulation tool. In Phase 3, NodeSync will be expanded to include leader/node failure handling, network partitions, recovery and log synchronization, performance evaluation, testing, and documentation.

Progress Overview

NodeSync has steadily progressed, with major project architecture and implementation completed. The following have been implemented:

- Project architecture and CMake-based infrastructure.
- C-based DSA modules such as linked list, queue, and logs.
- We wrote the C++17 classes for nodes, Raft states, messages, storage, and the simulated network
- Raft leader election with majority voting works. -commerce commands are handled, and their log entries are replicated across nodes.

At the end of Phase II, the main architecture is established and the core modules have been connected for initial testing.
Implementation is done on the leader election, e-commerce command processing, message handling and the rudiments of replicated-log behaviour. In short these provide the basis for the rest of the fault-tolerance work. The coming milestone will be to put an end to the normal flow and see what transpires. It is necessary to check the cluster's reaction to a node crash: does a new leader get put in if called for? How does a node that comes back online make up for committed entries? We intend to run through full scenarios and take our performance measurements before putting any results in a report.

Codebase Information

The codebase follows a modular architecture that separates node management, consensus handling, message processing, network simulation, storage, and application logic.

C++ handles the node and Raft logic, message objects, simulated network, and storage. The C layer provides the linked list, queue, and log structures used to organise messages and replicated entries. CMake is used to build the project.

Since messages are passed between nodes in the same process, the simulator does not currently establish real network connections.

The key remaining development work is the integration of failure scenarios, recovery and log catch-up, and end-to-end testing.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
DSA component testing	Pass	The linked list, queue and Log structures all work as expected
Node and state management	Pass	Nodes behave correctly and they move between states as they should
Message handling	Pass	Simulated messages are sent and received correctly
Load Balancer Testing	Pass	Requests are routed correctly Full-system integration is still to be checked
Raft Leader Election and Consensus	Pass	Leader election, heartbeats and majority commit all work
Log Replication and Storage	Pass	Logs replicate and committed operations are applied the same way on every node
Order and Inventory Operations	Pass	Orders and stock updates match across the replicated nodes
Failure Handling and Recovery	Pending	Still to test- node crashes, leader re-election, network partitions and log sync
End-to-End Integration and Performance	Pending	Still to run-full scenarios, recording election time, consensus time, recovery time and message count

Deliverables Progress

We have put the finishing touches on most of our Phase 2 plans. The project now has a CMake build in place and the requisite C and C++ architecture, which includes the C linked list, queue and log as well as the C++ classes for storage, messages, Raft states, nodes and the simulated network. In addition to that we have leader election, majority consensus, e-commerce command processing and majority-based log replication all running, along with the simulated communication.

What remains is the fault-tolerance side of things. We are currently at work on the consensus process and on simulating network partitions and the failure of leaders and nodes. Once that is done, the focus will shift to node recovery and log synchronization to see how they hold up. Testing and the like are being handled in parallel. We have set aside the finalization of the console-based distributed consensus simulator for after the other components are complete.

Finalization of the console-based distributed consensus simulator is scheduled after the accomplishment of the rest of the components.